

Sectiunea 1 – feedback:

- In cadrul acestui proiect, consider ca am reusit sa dobandesc cunostinte avansate in legatura cu programarea concurenta si, totodata, despre complexitatea agregatoarelor de stiri.
- Initial, am abordat problema punand accentul aproape in mod egal intre procesarea secventiala si cea paralela, insa am constatat rapid ca trebuie sa folosesc cat mai mult procesarea paralela pentru a avea o scalabilitate cat mai optima, respectiv sa mut cat mai multa logica de procesare in interiorul thread-urilor de tip Worker – in cazul meu, NewsWorker este responsabil pentru acest lucru.
- O reala provocare a fost logica de eliminare a duplicatelor, in special atunci cand doua thread-uri detecteaza un conflict de titlu. Insa, am rezolvat problema folosind un ConcurrentHashMap – seenTitles – si *blacklistedTitles* pentru a marca elementele invalide.
- Tinand cont de faptul ca, pe parcursul temei, am refactorizat o buna parte din implementare pentru a obtine o scalabilitate cat mai ridicata, si debugging ul pentru testele mari, estimez ca durata totala a implementarii a durat in jur de 15-20 de ore.

Sectiunea 2 – strategia de paralelizare:

- ***Explicarea modului in care este impartita munca intre thread-uri:***
 - Pentru a utiliza resursele cat mai eficient cu putinta, am optat pentru o distribuire dinamica a sarcinilor, deoarece o impartire statica care este prestabilita de la inceput ar fi dus la dezechilibre majore.
 - In implementarea mea, fiecare thread – NewsWorker – functioneaza ca un consumator autonom. Acestea preiau, pe rand, indexul urmatorului fisier disponibil dintr-o variabila partajata de tip *Atomic Integer*. Astfel, in momentul in care un thread termina de procesat un fisier mai mic, va prelua imediat altul, nemaifiind nevoit sa astepte dupa alte thread-uri.
 - Fluxul de executie al fiecarui Worker este impartit in 2 faze importante, separate de o bariera *CyclicBarrier* pentru a ma asigura de corectitudinea datelor in contextul eliminarii duplicatelor.
 - Prima faza o reprezinta citirea JSON-urilor de catre thread-uri si parsarea articolelor. Apoi, are loc verificarea unicitatii, folosind structuri concurente. Astfel, articolele care trec de aceasta filtrare sunt salvate intr-o lista locala a thread-urilor – *myCandidates* -.
 - Dupa ce toate thread urile asteapta la bariera pana cand toate fisierele au fost citite si toate conflictele de duplicare au fost rezolvate, urmeaza etapa a doua. Aici, doar pentru articolele valide, thread-urile executa operatiile de clasificare pe categorii si limbi, respectiv procesarea textului pentru cuvintele cheie si rapoarte statice.
- ***Descrierea mecanismelor de sincronizare folosite:***
 - Am optat pentru folosirea structurilor de date de tip concurente si primitive atomice in detrimentul blocurilor brute synchronized – care ar fi serializat executia -, pentru a

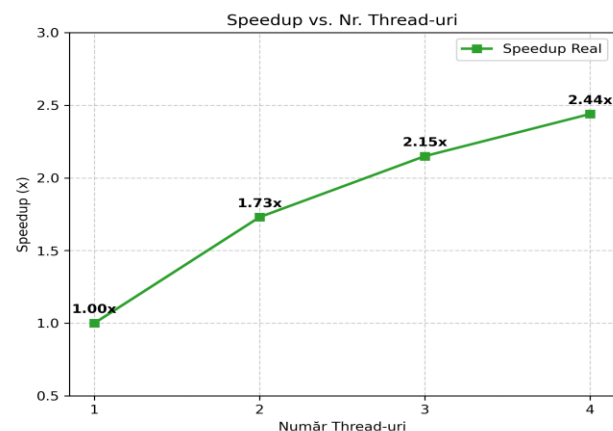
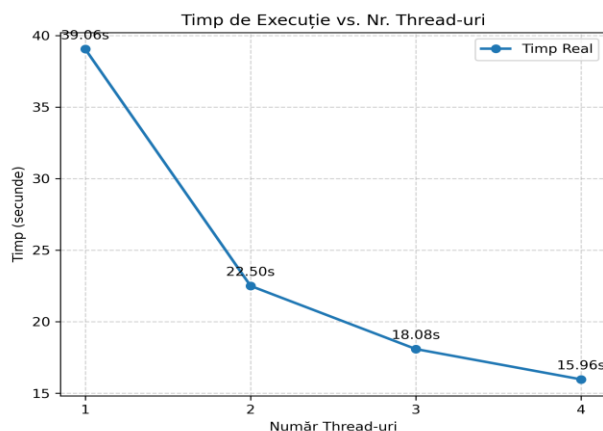
maximiza performanta paralela si consistenta datelor. Principalele mecanisme de sincronizare folosite sunt:

- **Variabilele atomice** – Atomic Integer- precum *fileIndex* sau *duplicatesCount*. Metoda `getAndIncrement()`, in contextul folosirii *fileIndex*, asigura ca distribuirea fisierelor se face atomic, eliminand riscul ca 2 thread-uri sa proceseze acelasi fisier, de exemplu race condition la citire, si garantand o incarcare echilibrata a thread-urilor.
- **Colectii concurente (ConcurrentHashMap):**
 - Utilizata pentru stocarea articolelor valide – *validArticles* - , respectiv a titlurilor – *seenTitles* –
 - Am ales aceasta structura datorita faptului ca folosesc mecanisme de blocare la nivel de bucket, permitand astfel accesul simultan a mai multor thread-uri fara a fi blocat map-ul.
- **Operatii atomice pe HashMap:**
 - De exemplu, `validArticles.putIfAbsent(uuid, article)` realizeaza verificarea existentei articolului si inserarea intr-un singur pas atomic. Acest lucru este esential in logica de duplicate, deoarece primul thread care apeleaza metoda este cel valid, urmatorul primind un rezultat non-null, semn ca a fost duplicat. De asemenea, folosirea lui `merge` in `globalKeywordCounts.merge(word, 1, Integer::sum)` ma ajuta sa incrementez thread-safe contoarele fara a necesita un lock explicit.
- **Bariera de sincronizare CyclicBarrier:**
 - Garanteaza faptul ca niciun thread nu va incepe sa calculeze statistici sau sa clasifice articole pana cand procesul de duplicate este terminat de catre toate thread-urile, asigurandu-ne ca lucram pe un set de date corect.
- **Cozi Thread-Safe – ConcurrentLinkedQueue:**
 - Utilizate ca valori in ConcurrentHashMap-uri pentru categorii si limbi, permitand workerilor sa adauge UUID-uri in lista aceleiasi categorii sau limbi, intr-o maniera non-blocanta.
- **Argumentare corectitudine si eficienta in contextul programarii paralele:**
 - **Corectitudinea solutiei** este asigurata de eliminarea completa a situatiilor de race condition in punctele critice, precum:
 - **Atomicitatea deciziilor** in contextul folosirii metodei `putIfAbsent` asigura ca, indiferent de ordinea de planificare a thread-urilor, un singur thread va reusi sa insereze un articol, in timp ce toti ceilalti vor detecta duplicatul. De asemenea, bariera `CyclicBarrier` previne propagarea datelor invalide catre structurile finale.
 - **Paralelizarea operatiilor costisitoare** a fost esentiala pentru performanta programului – de exemplu, mutarea procesarii textului in interiorul clasei *NewsWorker* -, executia ei distribuita oferind un speedup liniar semnificativ fata de o abordare secventiala pe firul principal.

- **Distribuirea dinamica a fisierelor**, folosind un AtomicInteger, elimina timpii morti. Astfel, se asigura faptul ca thread-urile care termina de procesat fisiere mai mici nu asteapta, ci preiau imediat noi sarcini.
- **Minimizarea contentiei** prin folosirea ConcurrentHashMap – pentru acces rapid la structuri-, si a ConcurrentLinkedQueue –pentru insertii non-blocante in listele de categorii/limbi – permit thread-urilor sa scrie rezultate simultan.
- **Sectiunea 3 – Analiza de performanta si scalabilitate:**
 - Pentru o analiza cat mai relevanta, am ales sa generez rezultatele folosindu-ma de test_3, fiind suficient de mare pentru o analiza precisa.
 - Setup:
 - Configuratia sistemului:
 - CPU: 12th Gen Intel(R) Core(TM) i5-1235U
 - Numar de nuclee: 6
 - RAM: 7.6Gi
 - Sistem de operare: Windows 10 (ruland Ubuntu 20.04 via WSL2)
 - Versiune de Java folosita: OpenJDK21
 - Dimensiunea datasetului: test_3 care contine 8273 de articole.
 - Rezultate: In tabelul de mai jos, au fost introduse datele pentru rulara secventiala, realizata de 3 ori, apoi i-a fost calculata media, respectiv pentru 2, 3, 4 thread-uri au fost calculate urmatoarele caracteristici: timpii individuali, media timpilor, speedup-ul si eficienta.

Nr. Thread-uri	Timpul Run 1 (s)	Timpul Run 2 (s)	Timpul Run 3 (s)	Media (s)	Speedup	Eficienta
1	38.02	39.98	39.17	39.056		
2	22.16	22.22	23.12	22.5	1.73x	0.86
3	18.07	18.19	17.99	18.083	2.15x	0.71
4	15.98	16.10	15.81	15.963	2.44x	0.61

- Graficele de mai jos, puse comparativ unul langa celalalt, intaresc ideea ca adaugarea de resurse de calcul, respectiv thread-urile, contribuie la reducerea timpului de executie.



- **Analiza si concluzii:**

- **Interpretarea rezultatelor:**

- Pe baza testelor efectuate pe setul de date *test_3*, observam faptul ca, odata cu cresterea gradului de paralelism, timpii de executie sunt clar imbunatatiti, insa scalabilitatea nu este perfect liniara.
 - Un punct cheie al analizei il reprezinta trecerea de la 1 la 2 thread-uri, deoarece scoate in evidenta castigul de performanta, reducand timpul de executie de la ~39s la ~22.5s. De asemenea, obtinem un speedup de 1.73x si o eficienta ridicata de 86%, demonstrand faptul ca implementarea paralelizeaza cu succes partea majora a procesarii.
 - Cand facem trecerea de la 3 la 4 thread-uri, performanta incepe sa creasca, insa curba de accelerare se aplatizeaza: adaugarea celui de-al 4-lea thread aduce un castig de doar ~2s fata de configuratia de 3 thread-uri, eficienta scazand la 61%.

- **Limitari si factori de influenta:** Speedup-ul la 4 thread-uri de 2.44x poate fi explicat de urmatoorii factori:

- Bariera CyclicBarrier poate fi considerat un overhead de sincronizare, deoarece timpul total este dictat de cel mai lent thread. Astfel, daca un thread primeste un fisier JSON mai mare sau mai complex, celelalte 3 thread-uri trebuie sa astepte la bariera inainte de a trece la categorii, limbi, cuvinte de interes, rapoarte statice, generand timpi morti.
 - Totodata, la 4 thread-uri, toate incearca sa scrie in ConcurrentHashMap-uri, astfel poate aparea contentia de memorie. Desi blocarea este la nivel de bucket, costul de coordonare a cache-ului procesorului creste.

- **Numarul optim de thread-uri pentru sistemul nostru:**

- Pentru sistemul pe care s-a realizat testarea si tinand cont de datasetul utilizat, consider ca numarul optim de thread-uri este 4, deoarece aceasta configuratie ofera cel mai scurt timp de executie (15.96s), chiar daca eficienta per thread este mai scazuta decat la rularea cu 2 thread-uri. Tinand cont de faptul ca dorim sa obtinem rezultatul cat mai rapid cu putinta, configuratia pe 4 thread-uri este cea mai performanta.